Parallel Chess Search with MPI, OpenMP, and GPU Offload

Jia Siqi (A0333724H) Jia Zeyue (A0326500U)

DSA5208 Project 3 — draft, 4 November 2026

Abstract

We study fixed-depth chess search as an irregular parallel workload. A serial negamax implementation with alpha-beta pruning is compared with OpenMP root splitting, deeper MPI+OpenMP task frontiers, a seeded-root variant that retains more pruning, and OpenMP GPU offload. The same legal-move library and simple material evaluation are used throughout. On Vanda, a depth-8, split-depth-3 task search visited 929.9 million nodes. A one-worker run took 198.1 seconds; the ten-node, 720-core run took a median of 1.754 seconds across three trials, a speedup of 112.95 with 15.7% efficiency. The serial alpha-beta baseline visited only 25.6 million nodes and took a median of 5.34 seconds. These contrasting baselines reveal that increasing parallelism can lose so much pruning that hardware speedup over the serial algorithm remains modest. An A40 GPU executed two offload strategies correctly: full-tree construction and on-demand chunks. At depth five, chunking reduced the median GPU time from 1.657 to 0.427 seconds, but both remained slower than serial; the depth-eight chunked run took 154.5 seconds.

1 Problem and algorithm

1.1 What a chess search computes

A chess position records the pieces on 64 squares, the player to move, castling rights, an en-passant square, and counters needed for the draw rules. At each position the player has a set of legal moves. Applying a move changes the position and hands the turn to the opponent. Repeating this process yields a game tree: a node is a position and an edge is a legal move. The branching factor varies greatly with the position. Moves may also create checkmate, stalemate, or a draw. We use Forsyth–Edwards notation (FEN) to specify reproducible starting positions.

Our program does not aim to play competitive chess. It searches to a fixed ply depth d, where one ply is one move by one player. At a nonterminal depth-zero leaf, it scores the material balance using pawn = 100, knight = 320, bishop = 330, rook = 500, and queen = 900. The king is not assigned a material value because it cannot be captured. Positive scores favor the side to move. Checkmate has a large negative score adjusted by distance to mate; stalemate and supported draw conditions score zero. A one-pawn advantage is thus roughly 100 score units. The leaf evaluation ignores king safety, positional value, quiescence, and opening knowledge. A score or selected move should therefore be read as a result of this experimental search, not as a claim about the best move in chess.

1.2 Minimax, negamax, and alpha-beta pruning

Chess is a two-player zero-sum game: a gain for the player to move is a loss for the opponent. A minimax search assumes that both players choose the best available continuation. Negamax

writes the same recurrence with one sign change at each turn:

$$V(s,d) = \begin{cases} E(s), & d = 0 \text{ or } s \text{ is terminal}, \\ \max\limits_{m \in M(s)} -V(s_m, d-1), & \text{otherwise}, \end{cases} \tag{1}$$

where s_m is the position after legal move m. Terminal outcomes are handled before the material evaluation. The root returns a score and the first legal move attaining the maximum. A simple exhaustive minimax backend serves as an independent search-control reference at shallow depth: it uses the same legal moves and evaluation but disables pruning.

Alpha-beta pruning avoids searching continuations that cannot affect the choice above them [1]. The search carries a lower bound α and an upper bound β. A child is searched with the negated window $[-\beta, -\alpha]$; if $\alpha \geq \beta$, later siblings cannot improve the relevant ancestor and are skipped. The returned value may be a bound when a cutoff occurs. Move order matters: a strong move discovered early can make many later branches cheap, while a poor first move leaves a wide window. Our baseline uses the legal-move order supplied by the library, without a transposition table or learned ordering. This makes the dependency between sibling searches visible in the parallel experiments.

2 Implementation and design decisions

2.1 Serial reference and third-party boundary

The project implements search, evaluation, task construction, MPI/OpenMP scheduling, GPU tree traversal, and counters in C++17. It vendors the unmodified, MIT-licensed single-header `Disservin/chess-library` revision `2a608d58f44776909bb6e2c09505e97ba1451319` [2]. That library supplies board representation, FEN parsing, legal move generation, move application and reversal, draw helpers, and UCI move formatting; it does not supply our search algorithm. The instructor approved this credited external dependency. The header and license are in `third_party/chess-library/`, with provenance in `THIRD_PARTY.md`. All calls to this host-oriented library, including legal move generation, occur on the CPU. The OpenMP device code is project code and contains no third-party chess-library calls.

2.2 Independent root and deeper-frontier tasks

The simplest CPU parallel method gives each legal root move an independent full-window search. An OpenMP dynamic loop schedules the root moves among threads; for MPI, moves are assigned to ranks by their index modulo the rank count. Each task owns a board copy and private search counters. Afterward, MPI combines candidate scores with `MPI_Allreduce` and sums counters. The search time reported for an MPI job is the maximum rank elapsed time, including the local frontier construction, search, and reductions, but not process launch or compilation.

A root may have only tens of moves, so root splitting cannot fill hundreds of cores. We also materialize the game tree to split depth two or three. Each frontier board becomes a task and performs an independent full-window negamax search for the remaining depth. At the initial position, split depth three produced 9,323 frontier nodes, enough tasks to distribute across 720 requested workers. The internal frontier nodes are combined bottom-up to recover the exact minimax root result. This exposes more parallelism at a cost: sibling tasks do not share alpha-beta bounds, so they can visit many nodes that serial alpha-beta would prune. Each MPI rank currently builds the same frontier. The assignment of tasks to ranks is static; OpenMP schedules the rank's tasks dynamically.

2.3 Seeded-root alternative

To retain more pruning, a second root method first searches the initial legal move with a full window. Its score a is broadcast to the MPI ranks. The remaining root moves use a as a fixed bound. A task that fails to beat a may return an inexact bound, but it cannot change the root maximum; a task that beats it supplies the needed value. This variant is implemented as `omp-pv` and `mpi-pv`. It exposes only the remaining root moves, and its static modulo rank assignment can be badly unbalanced. We measure the seed time, seed node count, and per-rank parallel work to examine this tradeoff.

2.4 OpenMP target feasibility study

The principal hybrid implementation uses MPI between nodes and OpenMP CPU threads within each node. Following the instructor's suggestion, we also explored OpenMP target offload as an additional experiment. Move generation is branch-heavy and uses a complex CPU board type. For the first GPU experiment, the CPU expands the legal tree to the chosen shallow depth and packs nodes and leaf board states into flat arrays. An OpenMP `target teams distribute parallel for` region gives independent root branches to the device [4]. Each device worker runs an iterative stack-based alpha-beta traversal of its branch and evaluates visited leaves. Pruning on the device cannot recover the cost of nodes already generated on the CPU. Generating small batches on the CPU instead would introduce repeated device launches and transfers; batching transfers and retaining device data are standard GPU performance considerations [5]. To reduce wasted generation, a second backend retains CPU alpha-beta above a fixed-depth chunk. Only when the CPU search reaches a chunk does it expand that subtree and offload its immediate children. The GPU searches each child with its own full window; the CPU combines exact child scores and continues pruning above the chunk. This preserves some cross-subtree pruning but adds one transfer and target launch per reached chunk. The library still executes only on the CPU. These are bounded offload studies: CPU tree expansion, device transfer, and limited branch parallelism remain measured costs. We require `OMP_TARGET_OFFLOAD=MANDATORY` and check `omp_is_initial_device()` to reject silent host fallback.

3 Experimental method

Runs used NUS Vanda PBS allocations, GCC 14.2.0 and Open MPI 5.0.7 for CPU jobs, and NVHPC 24.7 with an A40 device for GPU jobs. The CPU script requests one MPI rank per node and sets the indicated number of OpenMP threads per rank. We initially discovered that Open MPI's default policy bound a four-thread rank to one core. A controlled depth-7 rerun on the same node fell from 6.23 to 2.43 seconds when `-bind-to none` was used [3]. A separate PBS affinity probe showed that the unbound rank remained inside the four-CPU PBS cgroup. All corrected scaling runs use this launcher setting. PBS `place=scatter` and logged hostnames verify distinct nodes for multi-node jobs.

The main strong-scaling workload is the initial-position FEN, depth eight, split depth three. It visits exactly 929,868,807 nodes in every measured task-decomposition run. We repeat each 72-, 144-, and 288-core allocation three times inside one PBS job and use the median internal search time. The one-worker decomposition run has one trial; the 720-core configuration has three trials in a single PBS allocation. Source SHA-256, PBS job IDs, nodes, toolchain libraries, scores, and raw output are preserved in `experiments/raw/`. The independent serial alpha-beta baseline uses the same initial position and depth but performs less work because it prunes across siblings. We report both baselines rather than interpreting them as the same scaling reference. The exact main input is

```
rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
```

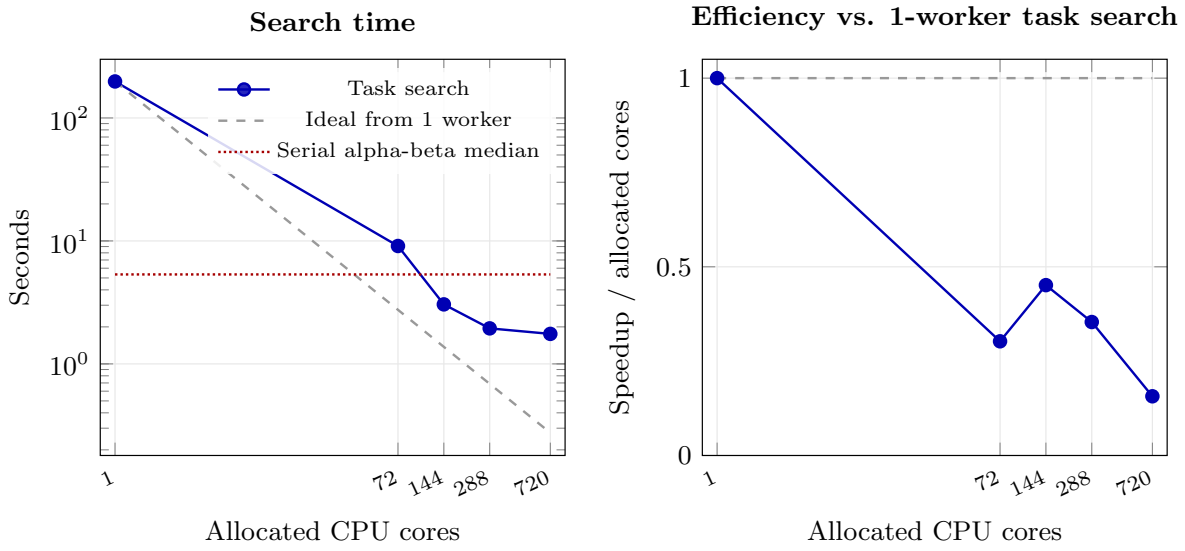
4 Results and interpretation

4.1 Strong scaling of the fixed task search

For p allocated CPU cores, we define speedup against a one-worker run of the *same task-decomposition algorithm* as $S(p) = T_{\text{task}}(1)/T_{\text{task}}(p)$ and efficiency $E(p) = S(p)/p$. Table 1 and Figure 1 show internal search time, excluding MPI startup. The one-worker baseline has only one trial, which limits the precision of the derived speedups.

Table 1: Initial position, depth 8, split depth 3. Search work is 929,868,807 nodes for every row. One worker has one trial; all other rows show medians of three trials in one allocation.

Cores	Hosts	Time (s)	$S(p)$	$E(p)$
1	1	198.129	1.00	100.0%
72	1	9.094	21.79	30.3%
144	2	3.048	65.01	45.1%
288	4	1.945	101.86	35.4%
720	10	1.754	112.95	15.7%



Initial position, depth 8, split depth 3. Search time excludes MPI startup.
1 worker: 1 trial; 72–720 cores: median of 3 trials per point.

Figure 1: Vanda internal search time and efficiency. The dashed time curve is ideal scaling from the one-worker task search; the dotted line is the serial alpha-beta median. The 720-core point is the median of three trials.

The 72-to-144-core median ratio is 2.98 and the 72-to-288-core ratio is 4.68, both above ideal ratios. We do not attribute this to a particular mechanism: allocation-specific node performance, cache effects, and the static rank distribution are plausible contributors and were not isolated in this series. Efficiency relative to the one-worker task search is only 15.7% at 720 cores. An earlier single-trial 720-core PBS feasibility job consumed 53 seconds of walltime although its internal search reported 1.593 seconds; process launch, setup, and teardown matter for a complete end-to-end cost.

The serial alpha-beta search visited 25,604,258 nodes and took 5.33762, 5.34012, and 5.34224 seconds (median 5.34012). The task decomposition visits about 36.3 times as many nodes. Thus the 720-core median is only 3.04 times faster than serial alpha-beta, despite using 720 allocated cores. This is the central algorithmic cost of parallelizing branches without their shared bounds.

4.2 Root bounds versus exposed parallelism

On the initial position at depth eight, the seeded-root method visited the same 25,604,258 nodes as serial alpha-beta. Its medians were 3.010, 2.853, and 4.205 seconds on one, two, and four four-thread nodes. Different node types were assigned to these allocations, so the times are exploratory. The instrumented four-node rerun on the same four CN hosts as the earlier four-node job spent only 0.347 seconds on the seed move; afterward rank 0 searched 14.14 million nodes for 3.459 seconds, while the other ranks searched 2.64–3.88 million nodes for 0.333–0.444 seconds. Static rank assignment, rather than the seed search alone, dominated that critical path. The comparison demonstrates that retaining pruning can reduce total work while too few or poorly distributed tasks limit scaling.

A second, move-rich Kiwipete FEN at depth six checks whether the initial position's result generalizes. Both serial and two-node seeded-root runs selected `e2a6` with score −90. Their respective node counts were 19.23 and 26.89 million and median times 5.359 and 4.670 seconds. The seeded-root method did more work here because its fixed initial bound was weaker than the serial search's evolving root bound. Its two ranks received 6.90 and 19.39 million parallel nodes, another example of imbalance. The exact Kiwipete input is recorded in `experiments/pv/README.md` and the corresponding raw PBS logs.

4.3 GPU execution and cost

All A40 jobs compiled and returned `ran_on_device=1`; GPU and serial scores and selected moves agreed at depths three through eight. Table 2 separates the CPU tree build plus target path total from time measured around the target regions. It does not separate device launch, transfer, and kernel execution.

Table 2: Initial-position A40 offload pilots, one trial each.

Depth	CPU-built tree nodes	GPU total (s)	Target (s)	Serial (s)
3	9,322	0.246888	0.244381	0.000125
4	206,603	0.270310	0.238880	0.000237
5	5,072,212	1.798000	0.660136	0.007303

The combined target-region time stayed near 0.24 seconds from depths three to four, then rose to 0.66 seconds at depth five. At depth five, CPU preparation alone took 1.138 seconds, about 156 times the serial search's 0.007303 seconds. The full GPU path took about 246 times as long as serial. The default two-million-node tree limit had to be raised to seven million for this bounded depth-five pilot. These workloads and the CPU-built full tree do not suit the GPU design; a larger GPU-native chess representation and a different task strategy would be needed for a useful performance comparison.

We then ran both GPU methods, with chunk depths two and three, three times each in one A40 PBS allocation. Table 3 reports medians. The logged tree-node count for chunks is cumulative over all generated chunks, not peak memory use. Every trial returned score 100 and move `b2b3`, matching serial. A chunk depth of three generated more nodes than depth two but used only 58 target launches instead of 844. It was faster in this comparison. The full-tree method generated 5.07 million nodes, while chunk depth three generated 0.62 million.

Table 3: Initial position, depth 5, same A40 allocation, median of three trials.

Method	Generated nodes	Chunk launches	Total (s)	Target (s)
Serial CPU	—	—	0.007177	—
GPU full tree	5,072,212	1	1.656630	0.560216
GPU chunks, depth 2	431,665	844	0.501165	0.419824
GPU chunks, depth 3	615,039	58	0.427455	0.310383

Chunk depth three also matched serial at depths six, seven, and eight in separate single-trial jobs. At depth eight it took 154.521 seconds versus 3.940 seconds for the paired serial run, generated 428.8 million nodes across 30,619 chunks, and spent 96.669 seconds in target regions. The CPU retained some pruning, but the repeated construction, transfers, and launches still dominated. Neither GPU method is competitive with the serial CPU search in these measured cases.

4.4 Separating data transfer from target execution

The original target-region timer includes transfers, launch, synchronization, and device traversal. We added separate profiling backends that use OpenMP device allocation and explicit `omp_target_memcpy` calls [4]. They time host-to-device (H2D) copies, the search target region, and device-to-host (D2H) copies separately. They also time CPU tree preparation and an initial device probe. The target-region timer still includes kernel launch and synchronization, so it is an *upper bound* on GPU computation time, not a pure arithmetic measurement. The explicit-copy path is a separate measurement from the implicit-map path above.

Table 4: A40 profiling phases in seconds. Depth-5 rows are medians of three trials in one allocation; depth 8 is one trial.

Method	Total	CPU build	Probe	H2D	Target search	D2H
Depth 5, full tree	1.304	0.762	0.149	0.035	0.358	<0.001
Depth 5, chunks 2	0.458	0.051	0.150	0.014	0.218	0.022
Depth 5, chunks 3	0.361	0.073	0.144	0.005	0.136	0.002
Depth 8, chunks 3	154.620	57.525	0.243	3.563	92.313	0.853

At depth eight, explicit transfers took 4.417 seconds, or 2.9% of total; CPU tree construction took 37.2%, and the search target regions took 59.7%. The 30,619 chunks uploaded 33.54 GB cumulatively and downloaded 17.8 MB. At depth five, the chunk-depth-three median spent 0.007 seconds on explicit copies, or 2.0% of total. Its 0.144-second device probe is a substantial one-time cost for this short process; the depth-eight run amortized it. The paired depth-five serial median was 0.004981 seconds: CPU tree construction alone was about 153 times that for the full-tree method and 15 times that for chunk depth three. At depth eight, CPU construction alone took 57.525 seconds versus 3.689 seconds for the complete serial search. CPU tree generation and the combined launch/search phase therefore dominate raw transfer time in these measured configurations. Device allocation and remaining host bookkeeping account for the small difference between the listed phases and total time.

5 Validation, reproducibility, and limits

The local smoke suite compares serial alpha-beta with exhaustive minimax, OpenMP root, seeded-root OpenMP, deeper OpenMP frontiers, and the GPU kernel on its host fallback path at depths one through three on seven FENs. These include checkmate, stalemate, promotion, and en-passant positions. All 21 shallow position/depth comparisons, the added depth-five chunk checks, and

the profiling host-fallback checks passed. On Vanda, MPI root/frontier and seeded-root runs matched the serial score on the initial position, the seeded-root MPI search also matched serial on Kiwipete, and all three A40 runs matched serial at their measured depths, including the new depth-eight chunk run, the three repeated depth-five comparisons, and the instrumented GPU profiles. Equal-score moves may legitimately differ; in the recorded comparisons the reported move also agreed. The selected move is always drawn from the library's legal-move list.

These checks validate search-control agreement under the shared move generator and evaluation; they are not an independent proof of full chess rules or of deep tactical strength. The source, vendored library and license, PBS scripts, raw logs, summary CSV, and figure source are shipped with this report. Run `make test` locally, or submit the documented PBS scripts after loading the recorded compiler modules. Logs identify source hashes and job allocations. GPU compilation needs NVHPC plus the GCC 13.2 toolchain; an initial failed compiler job is retained in the raw logs along with the successful reruns.

6 AI assistance and conclusion

OpenAI Codex assisted with design discussion, C++ implementation, MPI and OpenMP variants, GPU offload code, PBS scripts, SSH experiment execution, diagnosis, result summaries, documentation, and drafting this report and figure. The students chose the chess-search topic, supplied the assignment and instructor guidance, requested cluster experiments, and must review the code, measurements, interpretations, and final submission. We preserve raw Vanda outputs so reported measurements can be checked against actual jobs.

The study illustrates a tension specific to tree search. Splitting more deeply creates enough tasks for hundreds of cores, but destroys useful alpha-beta dependencies and inflates total work. Searching a promising root move first preserves pruning, yet exposes few tasks and suffers from static load imbalance. GPU offload is technically possible; on-demand chunking cuts generated work but remains slower than serial on these tests. The strongest future improvement would be a load-aware or dynamic MPI scheduler together with a parallel search scheme that shares safe alpha-beta bounds. Additional single-worker repetitions would improve the precision of the efficiency baseline.

References

[1] D. E. Knuth and R. W. Moore, "An Analysis of Alpha-Beta Pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975. https://doi.org/10.1016/0004-3702(75)90019-3.

[2] Disservin, *chess-library*, MIT-licensed C++ chess library, commit 2a608d58f44776909bb6e2c09505e97ba1451319. https://github.com/Disservin/chess-library.

[3] Open MPI, *mpirun / mpiexec*, version 5.0.7 documentation. https://docs.open-mpi.org/en/v5.0.7/man-openmpi/man1/mpirun.1.html.

[4] OpenMP Architecture Review Board, *OpenMP API Specification*, version 5.2, 2021. https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf.

[5] NVIDIA, *CUDA C++ Best Practices Guide*, section 10.1, "Data Transfer Between Host and Device." https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/.